Variables, types, and collections in Go

Contents

Deep reference for Go's value side: how data is declared, what types exist, and how arrays and slices behave.

1. Variables

Declaration forms

```
var x int                       // explicit type, zero value
var x int = 10                  // explicit type and value
var x = 10                      // type inferred from value
x := 10                         // short form (functions only)

var (                           // grouped
    name string
    age int = 30
    ok   bool
)
```

Top-level (package scope) variables use `var`. Short form `:=` only works inside functions.

Multiple declaration

```
var a, b, c int                 // all int, all zero
var a, b = 1, "two"             // different types
a, b := 1, "two"
a, b = b, a                     // swap, no temp variable
```

For function returns:

```
v, err := f()                    // both new
v, err = g()                     // both reassigned
v, err := h()                    // valid if at least one (here both) names exist -
        tricky
```

Shadowing

A `:=` inside a nested block creates a new variable that hides the outer one. This is the source of many silent bugs.

```
err := f()
if cond {
    err := g()       // shadows outer err
    log.Println(err) // logs the inner err
}
// outer err is unchanged
```

`go vet -shadow` catches obvious cases. The fix is to use `=` and declare ahead of time:

```
var err error
err = f()
if cond {
    err = g()
}
```

Blank identifier

`_` discards a value. Required when you can't ignore a return cleanly.

```
_, err := f()
for _, v := range slice { }
var _ Stringer = (*User)(nil)              // compile-time interface check
```

Package vs function scope

Package-level `var` is initialized before `main` runs, in dependency order, then by source order. Avoid relying on this; explicit init is clearer.

```
var defaultClient = http.Client{Timeout: 5 * time.Second}
```

2. Constants

Constants are values fixed at compile time. They can be numbers, strings, booleans, or characters.

```
const pi = 3.14159
const name = "alice"
const ok = true
const newline = '\n'

const (
    StatusOK    = 200
    StatusError = 500
)
```

Typed vs untyped constants

Untyped constants have a “default type” but can adopt any compatible type at use site. This is why `1` works as `int`, `int64`, `float64`, `byte`, etc.

```
const x = 10                    // untyped int constant
var i int = x                   // x becomes int
var f float64 = x               // x becomes float64

const y int = 10                // typed
var f float64 = y               // compile error: type mismatch
var f float64 = float64(y)      // explicit conversion needed
```

Prefer untyped constants. They compose better.

iota

`iota` is a counter inside a const block. Starts at 0, increments per line.

```
const (
     Red = iota     // 0
     Green          // 1
     Blue           // 2
)

const (                          // bitmask flags
    Readable = 1 << iota         // 1
    Writable                     // 2
    Executable                   // 4
)

const (                          // skip with _
    _ = iota
     KB = 1 << (10 * iota)       // 1024
     MB                          // 1048576
     GB                          // 1073741824
)
```

`iota` resets in each new const block. Within a block, it increments per line, not per identifier (so `A, B = iota, iota+1` counts once).

Constant expressions

Constants support arithmetic at compile time, including arbitrary precision.

```
const big = 1 << 100              // fine as a constant
var n = big                       // overflow: doesn't fit in int
var f = float64(big)              // OK
```

The compiler holds untyped numeric constants in unlimited precision and only narrows them at use.

3. Data types

Boolean

```
var b bool = true
!b; a && b; a || b
```

Zero value: `false`. No implicit conversion from numbers.

Numeric types

Family	Types	Notes
Signed	`int8`, `int16`, `int32`, `int64`, `int`	`int` is 32 or 64 bits, platform-dependent
Unsigned	`uint8`, `uint16`, `uint32`, `uint64`, `uint`, `uintptr`	`uintptr` holds a pointer-sized integer
Float	`float32`, `float64`	IEEE 754; default to `float64`
Complex	`complex64`, `complex128`	rare

Aliases: - `byte` = `uint8` - `rune` = `int32` (Unicode code point)

Overflow on integer arithmetic wraps silently (no panic). Division by zero panics for integers, produces `+Inf` / `NaN` for floats.

```
var x int8 = 127
x++          // becomes -128, no error
```

Type conversion

Go has no implicit numeric conversion. Every conversion is explicit.

```
var i int = 10
var f float64 = float64(i)
var u uint = uint(f)

var b byte = 'A'               // 65
var s string = string(b)       // "A", a single character

var n int = 65
var s string = string(n)     // "A", same as above (treats int as code point)
var s string = strconv.Itoa(n) // "65", numeric to string
```

The `string(int)` form is a code-point conversion, not a decimal one. Use `strconv` for that.

String

Immutable byte sequence. UTF-8 by convention.

```
s := "hello"
s := `raw
string with newlines`          // backticks: no escape processing

len(s)                         // byte length
s[0]                           // a byte
s + " world"                   // concatenation (allocates)
s == "hello"                   // value comparison
```

Strings are not arrays. `s[0]` returns a `byte`, not a `string`. Indexing past `len(s)-1` panics.

Conversion:

```
b := []byte(s)                 // copy bytes
r := []rune(s)                 // decode to runes (handles multi-byte)
s2 := string(b)                // bytes back to string (copy)
s3 := string(r)                // runes back to string
```

byte vs rune

```
var b byte = 'A'               // single byte, 0–255
var r rune = 'é'               // Unicode code point, may be multi-byte in UTF-8

s := "héllo"
len(s)                         // 6 (é takes 2 bytes in UTF-8)
utf8.RuneCountInString(s)      // 5 (actual character count)

for i, r := range s {        // decodes UTF-8 properly
    fmt.Printf("%d %c\n", i, r)
}
```

Type aliases vs new types

```
type UserID int                    // new named type (distinct from int)
type Email = string                // alias (interchangeable with string)

var u UserID = 10
var i int = int(u)                 // conversion needed

var e Email = "a@b.c"
var s string = e                   // no conversion needed (alias)
```

New types create distinct types that don't auto-convert. Aliases (with `=`) are just another name for the same type. Aliases are rare; named types are the common pattern for stronger typing.

Zero values

Every type has a usable zero value.

Type	Zero
Numbers	`0`
bool	`false`
string	`""`
pointer, slice, map, chan, func, interface	`nil`
struct	each field at its zero
array	each element at its zero

A nil slice can be ranged, appended to, and passed to `len`. A nil map is read-safe but write-panics. A nil channel blocks forever on send and receive.

4. Arrays

Fixed-size, value semantics. The size is part of the type.

```
var a [3]int                    // [0 0 0]
b := [3]int{1, 2, 3}
c := [...]int{1, 2, 3}          // size inferred
d := [5]int{1: 10, 3: 30}       // [0 10 0 30 0]

len(a)                          // 3 (always known at compile time)
a[0] = 5
for i, v := range a { }
```

Value semantics

Arrays are copied on assignment and on function call.

```
a := [3]int{1, 2, 3}
b := a                          // b is a full copy
b[0] = 99
// a is still [1 2 3]

func f(x [3]int) { x[0] = 0 } // modifies the copy, not a
```

This is why arrays are rare in Go. Slices wrap arrays with reference semantics.

Multi-dimensional

```
var grid [3][4]int              // 3 rows, 4 cols
grid[1][2] = 5

for r := range grid {
    for c := range grid[r] {
        fmt.Print(grid[r][c])
    }
}
```

Each row in `[3][4]int` is a `[4]int`, contiguous in memory.

Comparability

Arrays of comparable types are comparable element-by-element. Slices are not.

```
[3]int{1,2,3} == [3]int{1,2,3}       // true
```

This makes arrays usable as map keys when slices aren't.

```
type point [2]int
m := map[point]string{{0,0}: "origin"}
```

When to use an array

- The size is fixed and small (matrix dimensions, fixed-size buffers).
- You need value semantics (defensive copies for free).
- You need a map key with multiple components.
- You need a comparable composite.

In day-to-day code, the answer is “use a slice.”

5. Slices

A slice is a three-word header pointing at a backing array: pointer, length, capacity.

```
type sliceHeader struct {
    Data uintptr   // pointer to first element
    Len int        // current length
    Cap int        // capacity (from Data to end of array)
}
```

Declaration

```
var s []int                          // nil slice (no backing array)
s := []int{}                         // empty slice, non-nil
s := []int{1, 2, 3}                  // literal
s := make([]int, 5)                  // length 5, cap 5, zero-filled
s := make([]int, 5, 10)              // length 5, cap 10
s := make([]int, 0, 100)             // empty but with capacity hint
```

Prefer `make([]T, 0, n)` over `[]T{}` when you know the eventual size. It avoids reallocations during `append`.

Nil vs empty

```
var a []int                      // nil; len(a) == 0; a == nil
b := []int{}                     // not nil; len(b) == 0; b != nil
```

Both behave identically in `len`, `range`, and `append`. The difference matters only for JSON encoding (`nil` becomes `null`, empty becomes `[]`) and explicit nil checks.

len and cap

```
s := []int{1, 2, 3, 4, 5}
t := s[1:4]                          // [2 3 4]
len(t)                               // 3
cap(t)                               // 4 (from index 1 to end of backing array)
```

`cap` tells you how far you can grow without reallocating.

Aliasing

Slices share the backing array with any slice carved from the same source.

```
s := []int{1, 2, 3, 4, 5}
t := s[1:4]
t[0] = 99
// s is now [1 99 3 4 5]
```

This is the slice gotcha. Modifying through `t` modifies `s`.

append

`append` returns a (possibly new) slice. Always assign the result.

```
s := []int{1, 2, 3}
s = append(s, 4)                     // [1 2 3 4]
s = append(s, 5, 6, 7)
s = append(s, other...)              // spread another slice
```

If the underlying array has room (`len < cap`), `append` writes in place and bumps len. If not, it allocates a new larger array, copies, and returns the new slice.

Growth strategy: roughly 2x while small, 1.25x once large (implementation detail, may change).

The append-with-aliasing trap

```
a := []int{1, 2, 3, 4, 5}
b := a[:3]                           // [1 2 3], cap 5
b = append(b, 99)                    // writes into a's array, in place
// a is now [1 2 3 99 5]
```

This is one of the most surprising parts of slices. If `cap(b) > len(b)`, `append` mutates the shared backing array.

To force a fresh array, use three-index slicing or a `copy`.

Three-index slicing

```
s := arr[low : high : max]
```

`max` caps the resulting capacity. Subsequent `append` calls reallocate instead of mutating the source.

```
a := []int{1, 2, 3, 4, 5}
b := a[:3:3]                    // cap is forced to 3
b = append(b, 99)               // forces new array; a is unchanged
```

copy

`copy(dst, src)` copies `min(len(dst), len(src))` elements. Returns the number copied.

```
src := []int{1, 2, 3, 4}
dst := make([]int, len(src))
n := copy(dst, src)          // n == 4

// in-place delete at index i
s = append(s[:i], s[i+1:]...)

// in-place delete preserving capacity, no allocation
copy(s[i:], s[i+1:])
s = s[:len(s)-1]
```

Common slice patterns

```
// stack
stack = append(stack, x)
x, stack = stack[len(stack)-1], stack[:len(stack)-1]

// queue (inefficient at front; use container/list or a ring buffer for hot paths)
queue = append(queue, x)
x, queue = queue[0], queue[1:]

// reverse in place
for i, j := 0, len(s)-1; i < j; i, j = i+1, j-1 {
    s[i], s[j] = s[j], s[i]
}

// filter in place
n := 0
for _, v := range s {
     if keep(v) {
         s[n] = v
         n++
     }
}
s = s[:n]

// clear (Go 1.21+)
clear(s)                          // zeroes all elements

// slices package (Go 1.21+)
slices.Sort(s)
slices.Contains(s, 5)
slices.Index(s, 5)
slices.Reverse(s)
slices.Equal(a, b)
slices.Clone(s)                   // fresh backing array
```

Memory leak gotcha

A small slice holding a reference to a large backing array keeps the whole array alive.

```
func first10(s []int) []int {
    return s[:10]            // still references the original array
}
```

If `s` had 1 million elements, the returned slice keeps all 1 million alive. Fix by copying:

```go
func first10(s []int) []int {
    out := make([]int, 10)
    copy(out, s[:10])
    return out
}
```

Same applies to substrings: `s[:10]` keeps the original string alive. For long-lived references to short prefixes, copy.

Function parameter semantics

Passing a slice is cheap: just the 3-word header copies. The backing array is shared.

```go
func mutate(s []int) { s[0] = 99 }    // caller sees the change
func grow(s []int)   { s = append(s, 1) }   // caller does NOT see the change
```

The first works because both slices share the backing array. The second doesn't because `append` reassigns the local header.

To grow from a callee, return the new slice:

```go
func grow(s []int) []int { return append(s, 1) }
s = grow(s)
```

6. Best practices

1. Prefer untyped constants. They compose with any compatible type.
2. Use `:=` for new variables, `=` for reassignment. Don't accidentally shadow.
3. Initialize slices with `make([]T, 0, n)` when you know the size. Avoids repeated reallocation.
4. Always assign the result of `append`. `append(s, x)` alone is a bug.
5. Three-index slice when you need isolation from the source's capacity.
6. Copy on retention. If a long-lived structure holds a slice carved from a big array, copy first.
7. Use `slices` package functions instead of writing your own sort/contains/reverse (Go 1.21+).
8. Named types over aliases for stronger compile-time checks (UserID vs int).
9. Range strings for characters, index strings for bytes. Know which you need.
10. Don't compare floats with `==`. Use a tolerance: `math.Abs(a-b) < epsilon`.

7. Common gotchas

Gotcha	Symptom	Fix
`:=` shadows outer variable	Outer value unchanged	Use `=`; declare outer earlier
Forgot to assign append result	Slice doesn't grow	`s = append(s, x)`
Modifying sub-slice mutates source	Mystery data change	Three-index slice or copy
Nil map write	Panic	`make(map[K]V)` first
Comparing slices with `==`	Compile error	Use `slices.Equal`
`string(65)` for "65"	Returns "A"	`strconv.Itoa(65)`
`len(s)` for character count of UTF-8	Wrong number	`utf8.RuneCountInString`
Iterating bytes, expecting runes	Garbled multi-byte	Range over string
Holding small slice of huge array	Memory not freed	Copy into a fresh slice
Integer overflow	Silent wrap	Use a wider type or check first
Passing array to function	Whole array copied	Use a slice
Comparing floats with `==`	Flaky equality	Use a tolerance

8. Quick reference

What	Syntax
Declare with inference	`x := 10`
Declare without value	`var x int`
Constant	`const pi = 3.14`
Enum-like sequence	`const ( A = iota; B; C )`
Bitmask	`const ( R = 1 << iota; W; X )`
Type definition	`type UserID int`
Type alias	`type Email = string`
Numeric conversion	`float64(x)`
String to bytes	`[]byte(s)`
String to runes	`[]rune(s)`
Array literal	`[3]int{1, 2, 3}`
Array with inferred size	`[...]int{1, 2, 3}`
Slice literal	`[]int{1, 2, 3}`

What	Syntax
Empty slice with capacity	`make([]int, 0, 100)`
Slice with length	`make([]int, 5)`
Sub-slice	`s[1:4]`
Sub-slice capped	`s[1:4:4]`
Append element	`s = append(s, x)`
Append slice	`s = append(s, other...)`
Copy	`n := copy(dst, src)`
Delete at index i	`append(s[:i], s[i+1:]...)`
Length	`len(s)`
Capacity	`cap(s)`
Clear (Go 1.21+)	`clear(s)`
Sort (Go 1.21+)	`slices.Sort(s)`
Contains	`slices.Contains(s, x)`
Clone	`slices.Clone(s)`